RESUMELENS

Formal languages for resume processing

Victor Team | Estructuras Discretas III | Integrative Task 1 | 2026

01 Problem and objective

Resume text uses varied spellings and mixes technical qualifications with personal facts. Our objective is an executable, traceable pipeline that recognizes explicit qualification patterns and produces a structured candidate document.

Four stages, one contract

Regex / Python re

Extract skills, contacts, education, role mentions and total years.

FST / pyformlang

Normalize aliases and order tokens separately for each profile.

DFA / pyformlang

Execute four recognizers and retain state-transition traces.

DSL / textX

Generate and validate the complete document before rendering HTML.

Design rationale

React and Streamlit share one Python orchestrator. Unknown skills remain visible and do not satisfy requirements.

02 Formal models

Transducer: T = (Q, Sigma, Gamma, delta, omega, q0, F)
CLEAN and ALIAS map Node.js to NODE_JS. FILTER passes arrange categories. Positive years produce EXPERIENCE_PRESENT; exact years stay in the profile.

Recognizer: M = (Q, Sigma, delta, q0, F)
One DFA per profile. Strict violations reach a trap. Resume mode projects and orders evidence for each alphabet.

Profiles: Full Stack Developer; Machine Learning Engineer; Cloud & DevOps Engineer; Data Scientist.

Candidate language

Six ordered sections: **personal, contact, experience, education, skills, classification**. Repeated records preserve several jobs, qualifications and accepted profiles. Optional fields distinguish missing data from explicit zero.

Experience = "experience", "{",
[TotalYears], {WorkEntry}, "}";

textX uses a PEG parser. EBNF specifies structure; model processors check vocabulary, contact formats, nonblank text and duplicates. Validation of an edited declaration does not rerun qualification recognition.

03 Verification and findings

796 tests passed

Complete suite, 11 October 2026.
Python 3.12; pyformlang 1.0.11; textX 4.4.0.
React type checking and production build passed.

Mixed data skills: Python, Pandas, Matplotlib, Scikit-learn, SQL and Git accept both ML Engineer and Data Scientist.

Negative control: python@gmail.com cannot supply the Python skill. Malformed DSL reports its location and produces no HTML.

Preservation: repeated records, Unicode and optional fields survive a DSL round trip. HTML output escapes all candidate text.

Limits and next experiments

The system checks reported patterns, not professional competence. Negation and proficiency remain outside the model. Extraction does not reliably associate employers or dates; the DSL allows explicit completion.

Next: evaluate extraction precision and recall on an annotated corpus and expand context rules. Test counts are not accuracy or coverage percentages.

REFERENCES AND REPRODUCIBILITY

Romero (2021), Pyformlang, DOI: 10.1145/3408877.3432464. Dejanovic et al. (2017), textX, DOI: 10.1016/j.knosys.2016.10.023.
Python re: docs.python.org/3/library/re.html | textX grammar: textx.github.io/textX/grammar.html
Repository: github.com/JuanJoe-18/DiscreteIII-Integrative-Task-VictorTeam | Tests: python -m pytest tests -q